Here are in-depth answers to every question.

---

1. Define Salesforce, CRM, and Multi-Tenant Architecture

What is CRM? CRM stands for Customer Relationship Management. It is a centralised system — essentially a database of tables — that stores and manages all information and interactions a business has with its customers, prospects, and partners. Before CRM, companies kept leads in spreadsheets, support tickets in email, and marketing data in separate tools. No one had the complete picture. CRM solves this by bringing leads, accounts, contacts, deals, support cases, and activity history into one shared platform so every team — sales, service, and marketing — works from a single source of truth.

Scope of CRM:

- Sales — tracking leads, deals, forecasts, and revenue
- Service — managing support cases, SLAs, and knowledge
- Marketing — campaigns, campaign members, ROI tracking
- Analytics — reports, dashboards, KPIs
- Collaboration — activities, tasks, Chatter

What is Salesforce? Salesforce is a cloud-based CRM platform delivered entirely over the internet — there is nothing to install on your own servers. It is both SaaS (Sales Cloud and Service Cloud are ready-to-use applications) and PaaS (the underlying platform lets you build custom objects, automation, Apex code, and Lightning Web Components on top). Every customer accesses Salesforce through a browser, and their data is securely isolated even though they share the same physical infrastructure — this is multi-tenancy.

Multi-Tenant Architecture: In a multi-tenant model, one instance of the software runs on one set of servers, but it serves many customers (tenants) simultaneously. Each customer's data is logically separated by a unique Org ID — no tenant can see another tenant's data. Think of it like an apartment building: all residents share the same foundation, plumbing, and electricity infrastructure, but each flat is independent and private.

Why does this matter? Because resources are shared, Salesforce enforces Governor Limits — hard caps on SOQL queries, DML operations, CPU time, and heap per transaction — so no single tenant can monopolise shared resources and degrade performance for others. This is the fundamental reason governor limits exist, and it explains almost every constraint you encounter as a Salesforce developer.

---

2. Governor Limits

Governor Limits are per-transaction caps enforced by the Salesforce platform to protect the shared multi-tenant infrastructure.

Per-transaction limits (synchronous):

Limit	Synchronous	Asynchronous
SOQL queries	100	200
Records retrieved by SOQL	50,000	50,000
SOSL queries	20	20
DML statements	150	150
Records processed by DML	10,000	10,000
CPU time	10,000 ms	60,000 ms
Heap size	6 MB	12 MB
Callouts	100	100

The golden rule: Never put SOQL or DML inside a loop. A trigger receiving 200 records with a SOQL inside the loop = 200 queries = immediate LimitException.

LimitException is uncatchable. When you hit a governor limit, Salesforce throws this exception, terminates the transaction, and rolls back everything. You cannot catch it with try/catch — you must design code to stay within limits.

The fix — bulkification:

```
// WRONG
for (Account a : Trigger.new) {
    Contact c = [SELECT Id FROM Contact WHERE AccountId = :a.Id]; // 1 query per record
}


// CORRECT
Set<Id> accIds = new Set<Id>();
for (Account a : Trigger.new) accIds.add(a.Id);
List<Contact> cons = [SELECT Id, AccountId FROM Contact WHERE AccountId IN :accIds]; // 1 query
Map<Id, Contact> byAcc = new Map<Id, Contact>();
for (Contact c : cons) byAcc.put(c.AccountId, c);
```

---

3. Types of Relationships

Relationships connect objects so records can reference each other. There are seven types:

Lookup Relationship Loosely coupled. The child can exist without a parent. Deleting the parent does NOT delete the child. Child has its own owner and sharing settings. No roll-up summaries. Up to 40 lookups per object. The field is optional. Example: Contact → Account (a Contact can exist without an Account)

Master-Detail Relationship Strongly coupled. Child CANNOT exist without a master. Deleting the master cascade-deletes all children. The child inherits the master's owner and sharing. Roll-up summaries (COUNT/SUM/MIN/MAX) are supported. Max 2 master-detail fields per object. The field is always required. Example: Prescription → Case (a prescription cannot exist without its case)

Many-to-Many Not a direct relationship — built using a junction object. The junction has two master-detail fields, one to each parent. First master = primary (controls ownership/sharing/layout). Deleting either parent cascade-deletes junction records. Example: Student ↔ Course via Enrollment junction object

Self Relationship A lookup from an object back to itself. Used to model parent-child within the same object. Standard example: Account → Parent Account (parent-child company hierarchy). You can represent division, subsidiary, and ultimate parent all on the same Account object.

Hierarchical Relationship A special lookup available ONLY on the User object. Used to represent a reporting structure — Manager field on User points to another User. Salesforce uses this to traverse the role hierarchy.

External Lookup The parent is an external object (from Salesforce Connect, suffix __x). The child matches the parent via a field value rather than a Salesforce Record Id.

Indirect Lookup The child is an external object. The parent is a standard or custom Salesforce object. The match is done via an External ID field on the parent, not the Salesforce Id.

Key distinction for every interview: If the child CAN exist without a parent → Lookup (loosely coupled). If the child MUST have a parent → Master-Detail (strongly coupled).

---

4. Assignment Rule

An Assignment Rule automatically assigns a record to a specific user or queue based on criteria when the record is created or edited.

Available on: Lead and Case objects only (standard Salesforce).

How it works:

1. The rule has multiple entries evaluated in order (like an if/else chain)

2. The first entry whose criteria matches gets applied
3. Each entry specifies: the criteria + the assigned user/queue
4. If no entry matches, the default assignment applies

Lead Assignment Rule example:

- Entry 1: Lead Industry = Technology → Assign to Tech Sales Queue
- Entry 2: Lead Industry = Healthcare → Assign to Health Sales Rep
- Entry 3: Lead Annual Revenue > 1,000,000 → Assign to Enterprise Team
- Default: Assign to General Leads Queue

Case Assignment Rule:

- Entry 1: Case Priority = Critical → Assign to L3 Support Queue
- Entry 2: Case Type = Billing → Assign to Finance Support Team
- Default: Assign to General Support Queue

Key points:

- Only ONE assignment rule can be active at a time per object
- The rule fires on record creation by default; it can also be triggered by checking "Assign using active assignment rules" on update
- Assignment Rules run at Step 7 of the Order of Execution (after after-triggers) — this is why a trigger assigning a record can be overridden by an assignment rule if you're using after-insert

---

5. Case Escalation Rule

A Case Escalation Rule automatically escalates cases that have not been resolved or contacted within a specified time threshold.

Purpose: Ensures no case falls through the cracks. If a critical case sits open too long, it automatically re-routes to a senior agent, manager, or queue, and sends an alert email.

How it works:

1. The rule has criteria-based entries (like Assignment Rules)
2. Each entry has an "Age Over" threshold — how many hours the case can remain in a state before escalating
3. Age can be measured in business hours or calendar hours
4. Actions on escalation: reassign the case, send an email alert, or both

Example configuration:

Entry 1:

Criteria: Case Priority = 'Critical' AND IsClosed = false

Age Over: 4 hours (Business Hours)

Action 1: Reassign to Senior Support Manager

Action 2: Send email "Critical case unresolved — escalated"

Entry 2:

Criteria: Case Priority = 'High' AND IsClosed = false

Age Over: 8 hours

Action: Send email to Case Owner's manager

Key points:

- You can use Business Hours (only counts working time) or calendar hours
- Business Hours must be set up in Setup → Business Hours first
- Escalation is triggered by Salesforce's background escalation engine — it runs periodically, not in real time
- The clock starts from case creation or the last modification depending on configuration

---

6. Approval Process

An Approval Process is an automated workflow that routes a record to one or more users for formal review and approval. It enforces business rules, prevents unauthorised actions (like large discounts), and maintains a full audit trail of decisions.

When to use it: When a human must review and sign off on a record before an action is finalised — discounts above a threshold, contracts, expense claims, job offers, budget requests.

Full anatomy:

Component	Description
Entry Criteria	The conditions the record must meet to be eligible for submission
Approver	A specific user, the submitter's manager field, a queue, or the related user on a lookup

Component	Description
Approval Steps	One or more ordered review stages, each with their own criteria and approver
Initial Submission Actions	What happens when the user submits for approval
Approval Actions	What happens when a step is approved
Rejection Actions	What happens when a step is rejected
Recall Actions	What happens when the submitter recalls before a decision
Final Approval Actions	What happens when ALL steps are approved
Final Rejection Actions	What happens when any step is rejected

Example — Discount Approval:

- Entry Criteria: Opportunity Discount > 20%
- Step 1: Assign to Sales Manager. On approve → Status = Approved. On reject → unlock + notify.
- Final Approval Actions: Set Status = Final Approved, send congratulations email
- Final Rejection Actions: Set Status = Rejected, send rejection notification

Record locking: A record is automatically locked when submitted — no one can edit it during the approval process except the current approver (and admins). This is intentional to prevent gaming the system.

---

7. Initial Submission Actions

Initial Submission Actions fire the moment a user clicks the Submit for Approval button — before any approver has seen it. They are the very first step in the approval chain.

Available action types:

1. **Email Alert** — Send a notification email (e.g., "Your request has been submitted and is awaiting approval")
2. **Field Update** — Change a field value on the record (e.g., set Status = 'Pending Approval')
3. **Task** — Create a task for someone
4. **Outbound Message** — Send an XML message to an external system (rare)

Typical use:

- Lock the record (Salesforce does this automatically — the record becomes read-only)
- Set a "Submitted By" or "Submitted Date" field
- Email the submitter confirming receipt
- Email the approver(s) that something needs their attention
- Log an activity or create a follow-up task

Why they matter: Initial submission actions run BEFORE the approval step. So when the approver opens the record, they already see a locked record with Status = 'Under Review', giving them context about what they're reviewing.

---

8. Why Restrict a Picklist to Defined Values Only

When you create a picklist field, Salesforce gives you the option to "Restrict picklist to the values defined in the value set" (also called a validated picklist or "Use a global value set"). Here's why this matters:

The problem without restriction: If the picklist is not restricted, values can be imported via the Data Loader or API that aren't in the defined list. For example, if your Status picklist has values New, In Progress, Closed — but someone runs a Data Loader import with Status = "Completed" (not in the list), that value is silently accepted. Your reports, automation, and validation rules that check for specific picklist values then behave unpredictably.

Why restrict it:

1. **Data integrity** — only pre-approved values can exist in the field. No rogue values from imports or APIs.
2. **Automation reliability** — flows, validation rules, and triggers that check ISPICKVAL(Status, 'Closed') always work correctly because there are no unexpected values.
3. **Reporting accuracy** — GROUP BY and filter reports give clean, predictable groups.
4. **UI consistency** — users can only pick from the approved list, nothing else.

Formula context: You cannot directly compare a picklist with = in a formula. You must use ISPICKVAL(field, 'value') or TEXT(field) = 'value'. This is because picklists are internally stored as tokens, not plain strings. Restricting to defined values ensures those tokens are always valid.

---

9. Change Set

A Change Set is Salesforce's point-and-click tool for deploying metadata (configuration, not data) from one org to another. It packages customisations — objects, fields, page layouts, Apex classes, triggers, flows — and moves them between connected orgs.

Two types:

- **Outbound Change Set** — created in the SOURCE org; you add components and upload
- **Inbound Change Set** — received in the TARGET org; you validate and deploy

Deployment flow:

Developer Sandbox (build)

→ Outbound Change Set created here

→ Uploaded

→ Appears as Inbound Change Set in UAT Sandbox

→ Validated (runs all Apex tests)

→ Deployed to UAT Sandbox

→ Repeat for Production

What you can include: Objects, custom fields, page layouts, validation rules, flows, Apex classes, triggers, profiles (partial), permission sets, custom labels, email templates, reports, dashboards, and more.

What you CANNOT include: Data (records) — Change Sets move metadata, not records. Users, record-sharing rules, and some settings are also excluded.

Limitations:

- Orgs must be connected (Setup → Deployment Connections)
- Cannot delete components via a change set (only add/modify)
- No version control — you can't see what changed between change sets
- Slower than CLI/Metadata API for large deployments

Modern alternative: sf project deploy start using Salesforce DX + source control — preferred in CI/CD pipelines. Change Sets are suitable for admin-led, non-scripted deployments.

---

10. Roll-Up Summary

A Roll-Up Summary field lives on the master side of a master-detail relationship and automatically aggregates values from the child records below it.

Formal definition: A Roll-Up Summary field calculates a value — COUNT, SUM, MIN, or MAX — across all child records of the master, with an optional filter to restrict which child records are included.

Requirements:

- Must be on the MASTER object (not the detail)
- The relationship must be MASTER-DETAIL (not lookup)
- The field being aggregated must be on the DETAIL/CHILD object

Example:

Master: Invoice

Child: Invoice_Line_Item__c

Roll-Up Summary on Invoice:

Name: Total_Invoice_Amount__c

Function: SUM

Summarise field: Line_Item_Amount__c (on Invoice_Line_Item__c)

Filter: none

Result: Invoice.Total_Invoice_Amount__c = sum of all Line Item amounts

When it recalculates: Whenever a child record is inserted, updated, or deleted, Salesforce recalculates the roll-up on the parent. This can trigger the parent's validation rules and workflow, so be mindful of cascading effects.

---

11. Options Available on Roll-Up Summary

When creating a Roll-Up Summary field, you configure four main options:

1. Roll-Up Type (the aggregate function):

Function	Returns	Example
COUNT	Number of child records	Total number of orders on an Account
SUM	Total of a numeric child field	Total invoice value across all line items
MIN	Smallest value in a child field	Earliest Close Date across all opportunities

Function	Returns	Example
MAX	Largest value in a child field	Highest single payment amount

2. Summarised Object: The child object whose records you want to aggregate.

3. Field to Aggregate: For SUM/MIN/MAX, the specific numeric, currency, date, or date-time field on the child you want to aggregate. For COUNT, no field is needed.

4. Filter Criteria (optional): A condition that restricts which child records are included in the rollup. Example: only SUM Line Items where Status = 'Approved'. If no filter is set, all child records are included.

Practical example with all options:

Object: Account

Roll-Up: Total_Won_Revenue__c

Type: SUM

Child Object: Opportunity

Field: Amount

Filter: StageName = 'Closed Won'

Result: Total_Won_Revenue__c shows sum of Amount

for all Closed Won Opportunities on the Account

---

12. Types of Email Templates

Salesforce has four types of email templates:

1. Text Plain text only — no formatting, no images, no HTML. Universal compatibility — works with any email client. Best for: simple transactional notifications. Supports merge fields.

2. HTML with Letterhead (Classic) Uses a pre-designed letterhead (your company logo, brand colours, font) combined with the email body. The letterhead is created separately and reused across templates. Best for: branded marketing-style emails.

3. Custom HTML (HTML without Letterhead) You write raw HTML directly — full control over layout, styling, and design. No pre-existing letterhead constraint. Best for: highly customised HTML emails where you design every element. Requires HTML knowledge.

4. Visualforce Dynamically generated emails using Visualforce markup — can include conditional logic, iterate over related records, display data from multiple objects in one email.

Best for: complex, data-driven emails that can't be expressed with merge fields alone (e.g., an email that includes all open line items for an opportunity).

Where email templates are used:

- Approval Process actions (send notifications to approvers/submitters)
- Workflow Rules and Flows (send email alerts)
- Case Auto-Response Rules
- Lead Auto-Response Rules
- Manual email sends from records

Merge fields: All template types support merge fields like {!Account.Name}, {!Opportunity.CloseDate}, and {!User.FirstName} to personalise content dynamically.

---

13. Standard Objects — Examples

Standard objects are prebuilt by Salesforce and available in every org:

Object	Purpose	Key fields
Lead	Unqualified prospect — not yet a confirmed customer	Name, Company, Email, Phone, Industry, LeadSource, Status
Account	A company/organisation you do business with	Name, Type, Industry, AnnualRevenue, BillingAddress
Contact	An individual linked to an Account	FirstName, LastName, Email, Phone, AccountId
Opportunity	A sales deal in progress	Name, Amount, CloseDate, StageName, AccountId
Case	A customer support request or issue	Subject, Description, Priority, Status, ContactId
Campaign	A marketing initiative tracked for ROI	Name, Type, StartDate, EndDate, BudgetedCost
Product (Product2)	What the company sells	Name, ProductCode, Family, IsActive
Quote	A formal pricing proposal	Name, Status, TotalPrice, AccountId, OpportunityId
Contract	A business agreement	AccountId, Status, StartDate

Object	Purpose	Key fields
Task	A to-do action	Subject, DueDate, WhoId (Contact/Lead), WhatId (related record)
Event	A meeting or scheduled activity	Subject, StartDateTime, EndDateTime
User	A Salesforce login	Username, Email, ProfileId, IsActive, ContactId
Knowledge	An article in the knowledge base	Title, ArticleNumber, PublishStatus
Campaign Member	A junction between Campaign and Lead/Contact	CampaignId, LeadId/ContactId, Status

---

14. What Can You Control Using a Profile

A Profile is the fundamental access control object. Every user has exactly one profile and it controls:

1. Object Permissions (CRUD) Create, Read, Edit, Delete per object. Also View All and Modify All per object.

2. Field-Level Security (FLS) Per field on each object: Visible + Editable, Visible (Read-Only), or Not Visible (Hidden).

3. App Permissions Which Lightning apps are accessible and which is the default app.

4. Tab Visibility Default On (visible), Default Off (not on the nav bar), or Hidden (completely hidden from the user).

5. Page Layouts Which page layout a user sees per object per record type — typically configured via Record Type assignment but based on the profile.

6. Record Type Access Which record types a user can create and see for each object.

7. Login Hours The time windows within which the user is allowed to log in (e.g. Monday–Friday 9am–6pm). Attempts outside these hours are blocked.

8. Login IP Ranges The IP address ranges the user must be within to log in. Enforced in addition to login hours.

9. Password Policies Minimum length, complexity requirements, expiry days, and number of failed attempts before lockout.

10. System Permissions High-level powers like Modify All Data, View All Data, Manage Users, Author Apex, Manage Salesforce CRM Content, etc.

11. API Access and Connected App Policies Whether the user can access the Salesforce API, and policies for specific connected apps.

Key rule: Standard profiles (System Administrator, Standard User, etc.) cannot be fully modified. You must clone a standard profile and then customise the clone. Users cannot be deleted — only deactivated.

---

15. Permission Set

A Permission Set is an additive collection of permissions that grants users extra access beyond what their profile provides. It never removes permissions — it only adds. A user can have multiple permission sets simultaneously.

Why use permission sets:

- Instead of creating dozens of slightly different profiles, use a minimal baseline profile and layer specific permissions via permission sets
- Grant a single user access to one additional object without changing their profile (which would affect all users on it)
- Temporary access — assign a permission set for a project, remove it when done

What you can grant:

- Object CRUD permissions
- Field-level security (edit, read on specific fields)
- Tab visibility
- System permissions
- App permissions
- Apex class access
- VF page access

Permission Set Group: Bundles multiple permission sets into one unit for easy assignment — assign the group instead of assigning each permission set individually. A PSG can also "mute" specific permissions from member sets (remove a permission the set would normally grant).

Key difference from Profile:

Profile: Mandatory (1 per user); sets the FLOOR

Permission Set: Optional (many per user); raises the CEILING

---

16. OWD — All Options Explained

OWD (Organisation-Wide Defaults) set the default access level for records a user does NOT own. They define the most restrictive baseline — the floor below which no one can go (unless you use Restriction Rules to go even lower).

Options:

Private Only the record owner and users above them in the Role Hierarchy can view and edit the record. No one else has any access by default. Most restrictive. Used for sensitive data. Example: HR performance reviews — only the employee's direct manager and HR admin should see them.

Public Read Only Everyone in the org can view the record, but only the owner (and those above in the hierarchy) can edit it. Used when all users need visibility but editing must be restricted. Example: Product catalog — all sales reps can read products, but only Product Managers can change them.

Public Read/Write Everyone can view AND edit the record regardless of ownership. Used for low-sensitivity objects. Example: A shared training schedule object where anyone can update it.

Public Read/Write/Transfer *(Lead and Case only)* Everyone can view, edit, AND reassign ownership. Covered below in question 17.

Controlled by Parent Access is inherited from the master record. Available only on detail objects in a master-detail relationship. You cannot independently set OWD on a detail object — it always follows its master. Example: Invoice Line Items always follow the Invoice's access settings.

Full Access *(Campaign only)* Everyone can view, edit, delete, and transfer. Maximum access. Specific to Campaign.

---

17. Why "Public Read/Write/Transfer" Only on Lead and Case

This is a great observation. The "Public Read/Write/Transfer" OWD setting allows users to not just view and edit, but also TRANSFER OWNERSHIP of a record to another user or queue. This option exists only for Lead and Case because of specific business workflows on those objects.

Lead — why Transfer is needed: Leads are routinely reassigned as part of the sales process. A lead may come in through a web form, sit in a general queue, and then be claimed (transferred to themselves) by a sales rep. Lead nurturing often involves passing a lead from one rep to another. If ownership transfer were restricted, this collaborative flow would break down. Assignment Rules also transfer leads by design.

Case — why Transfer is needed: Support cases regularly move between agents, queues, and teams as they escalate or are resolved. A Tier 1 agent might transfer a case to Tier 2. A queue

member pulls a case from the queue (claiming ownership). Omni-Channel routes cases by reassigning them. All of these require the transfer capability to be broadly available.

Why not for Opportunity or Account? Opportunities and Accounts represent more stable, long-term relationships. Their ownership changes are deliberate and managed — usually by an admin or manager via Mass Transfer. Broad open transfer rights would be chaotic and a security risk (anyone could steal another rep's pipeline). So those objects don't get the Transfer option at OWD.

---

18. Types of Sandboxes

Salesforce offers four types of sandbox environments, each suited to different stages of the development lifecycle:

Type	Data	Refresh interval	Storage	Use
Developer	No production data (you create your own)	1 day	200 MB data / 200 MB files	Individual developer work, quick builds
Developer Pro	No production data	1 day	1 GB data / 1 GB files	Larger teams, more complex dev work
Partial Copy	A sample of production data (you configure the export)	5 days	5 GB data	Testing with representative real data
Full	A complete copy of all production data	29 days	Same as production	Full UAT, performance testing, final validation before go-live

Refresh means taking a fresh copy of production metadata (and for Partial/Full, data too). During refresh, all customisations in the sandbox are overwritten by the production state.

Which sandbox for which stage:

Development → Developer / Developer Pro Sandbox

Integration testing → Developer Pro / Partial Copy

UAT / Staging → Partial Copy / Full

Final validation → Full Sandbox

---

19. What is Profile, User, and Permission Set?

User: A User is a person who logs into Salesforce. A User record stores their identity, credentials, and settings. Key fields: Username (globally unique, email format), Email, ProfileId, IsActive, Role, ContactId (for portal users). Without a User record there is no login. A User must have exactly one License and one Profile. Users cannot be deleted — only deactivated.

Profile: A Profile is the mandatory access control definition for a User. Every user has exactly one profile. It sets the BASELINE for everything that user can access — object CRUD, field visibility, app and tab access, record type assignment, login hours, IP ranges, and password policies. Standard profiles can't be fully modified — clone them first.

Permission Set: A Permission Set is optional additional access granted on top of the profile. A user can have many. It only ADDS permissions — never removes. Used to give a specific user (or group of users) access to an object, field, or system capability that their profile doesn't provide, without modifying the profile for all users on it.

The mental model:

Profile = the floor (minimum, mandatory, everyone on this profile gets this)

Permission Set = a ladder (optional, individual, lift specific users higher)

Permission Set Group = a bundled set of ladders, assigned as one unit

---

20. Validation Rules — Examples Applied

Validation rules fire when a record is saved. If the formula returns TRUE, the save is blocked with your error message.

Rule 1 — End Date cannot be before Start Date:

End_Date__c < Start_Date__c

Error message: "End Date cannot be earlier than Start Date."

Rule 2 — Description required for Critical cases:

```
AND(
  ISPICKVAL(Priority, 'Critical'),
  ISBLANK(Description)
)
```

Error message: "Description is required for Critical priority cases."

Rule 3 — Valid phone format (10 digits only):

NOT(REGEX(Phone, "[0-9]{10}"))

Error message: "Phone number must be exactly 10 digits with no spaces or dashes."

Rule 4 — Close Date must not be in the past for open Opportunities:

```
AND(
  NOT(ISPICKVAL(StageName, 'Closed Won')),
  NOT(ISPICKVAL(StageName, 'Closed Lost')),
  CloseDate < TODAY()
)
```

Error message: "Close Date cannot be in the past for an open Opportunity."

Rule 5 — Email required when Opt-In is Yes:

```
AND(
  ISPICKVAL(Opt_In__c, 'Yes'),
  ISBLANK(Email)
)
```

Error message: "Email is required when the customer has opted in for communications."

Rule applicable in NeelamCare project — block case closure without prescription: This was implemented as a Trigger (not a validation rule) because checking "does a child record exist?" requires a SOQL query — which validation rules can't do. A trigger queries COUNT of Prescription records and calls addError() if zero.

---

21. One-to-One, One-to-Many, and Many-to-Many Relationships

One-to-One: One record in Object A relates to exactly one record in Object B, and vice versa. Salesforce doesn't have a native one-to-one relationship type, but you approximate it using a Lookup with a unique index — create a lookup field and mark the field as Unique. This ensures no two records in the child can point to the same parent, effectively enforcing one-to-one. Example: Each Employee has exactly one Employee_ID_Card, and each ID Card belongs to exactly one Employee.

One-to-Many: One record in Object A relates to many records in Object B, but each child has only one parent. This is the Lookup or Master-Detail relationship. It's the most common relationship type. Example: One Account has many Contacts. One Opportunity has many Opportunity Line Items. One Case has many Prescriptions.

Many-to-Many: One record in Object A relates to many records in Object B, AND one record in Object B relates to many records in Object A. Salesforce implements this with a Junction Object. The junction has two master-detail fields — one pointing to each parent. Example: Student ↔ Course. A student can enrol in many courses, and a course can have

many students. The junction object Enrollment has two master-detail fields — one to Student and one to Course. Each Enrollment record = one student in one course.

---

22. Junction Object — Definition and Purpose

Definition: A Junction Object is a custom object with exactly two master-detail relationships, one to each of the two parent objects, used to implement a many-to-many relationship between them.

Why we use it: Salesforce relationships are inherently one-to-many. You cannot directly link an Account to many Campaigns AND a Campaign to many Accounts in a single field. The junction object sits in the middle, holding one record per unique pair — it represents the association between the two parents.

Structure:

Parent A Junction Object Parent B

Student ←M-D— Enrollment __c —M-D→ Course

Id=1 StudentId = 1 Id=A

Id=2 CourseId = A

Enrollment record 2:

StudentId = 1

CourseId = B → Aryan enrolled in LWC Training

Enrollment record 3:

StudentId = 2

CourseId = A → Riya enrolled in Salesforce Admin

Primary vs secondary master: The FIRST master-detail created becomes the primary master. It controls:

- The junction record's ownership (inherited from primary master)
- The junction record's sharing settings
- The look of the junction's page layout

Cascade delete: Deleting EITHER parent cascade-deletes all related junction records. An enrollment without either a student or a course is meaningless, so this is the correct behaviour.

Real-world examples:

- Student ↔ Course (via Enrollment)
- Patient ↔ Outreach Program (via Program_Patient in NeelamCare)
- Campaign ↔ Contact/Lead (via Campaign Member — this is a standard Salesforce junction)
- Doctor ↔ Hospital (via Appointment)

---

23. OWD, Sharing Rules, and Restriction Rules

OWD (Organisation-Wide Defaults): OWD sets the BASELINE access for records a user doesn't own. It is the floor — the minimum everyone gets on other people's records. Options: Private, Public Read Only, Public Read/Write. If OWD is Private, no one outside the owner and their role hierarchy can see the record unless access is explicitly opened.

Sharing Rules: Sharing Rules open access BEYOND what OWD provides. They extend access sideways — to roles, public groups, or roles & subordinates — for records owned by certain people or matching certain criteria.

Two types:

- **Owner-based**: "Share records owned by users in [Role X] with [Public Group Y] at Read/Write"
- **Criteria-based**: "Share records where Region = 'South' with [Role: South Managers] at Read Only"

Key constraint: Sharing Rules can ONLY open access. They can never restrict below OWD. If OWD is Public Read/Write, sharing rules don't add anything — everyone already has full access.

Restriction Rules: Restriction Rules do the OPPOSITE — they tighten access. They filter what a user can see DOWN to a subset, even if OWD or sharing would normally give them broader access.

Example: OWD is Public Read/Write for Contact. HR staff should only see Contacts in their region. Create a Restriction Rule on Contact: HR Profile users can only see Contacts where Region = {user's own Region value}.

The full picture:

OWD = sets the floor (default access)

Role Hierarchy = opens access UPWARD (managers see subordinates' records)

Sharing Rules = open access SIDEWAYS (horizontal)

Teams = record-specific additional access

Manual Sharing = one-off record-level access

Restriction Rules = NARROWS access (below OWD level)

---

24. Different Types of Refreshes in Salesforce

Sandbox Refresh: Taking a fresh copy of production data and/or metadata into a sandbox. Refresh intervals vary by type: Developer (1 day), Developer Pro (1 day), Partial Copy (5 days), Full (29 days). After a refresh, all customisations in the sandbox are overwritten by the production state.

Data Refresh: Using Data Loader or the API to re-import updated records. Not a Salesforce platform feature — an external data operation.

Report Refresh: Running a report to fetch the latest data. Reports are not real-time — you click Refresh to pull current records matching the filters.

Dashboard Refresh: A dashboard shows a snapshot based on its last refresh time. You can manually refresh, schedule an automatic refresh, or subscribe to a scheduled email. Dynamic dashboards refresh based on the logged-in user's data each time they open it.

Page Refresh (browser): A simple F5 or browser reload. Pulls the latest record data from the server.

Wire Adapter Refresh in LWC: refreshApex(this.wiredResult) forces a re-fetch of wired Apex data, bypassing the client-side LDS cache. Used in LWC after a successful DML to show the user the updated data immediately.

Cache Refresh: Salesforce caches metadata (like Custom Settings data) per application session. Cache refresh happens on new deployments or when Salesforce clears the cache on its side.

---

25. Self Relationship of Account

The Account object has a special self-relationship called the Parent Account field. It is a Lookup field from Account back to Account — an Account can reference another Account as its parent.

Why it exists: Businesses operate in hierarchies — divisions, subsidiaries, and holding companies. You need to model:

Tata Group (Ultimate Parent Account)

└── Tata Consultancy Services (Parent Account)

└── TCS Chennai Division (Child Account)

└── TCS Chennai Delivery Unit (Grandchild Account)

What it enables:

- You can see the entire company hierarchy on any Account record
- The Account Hierarchy button shows the full tree
- Roll-up reporting can aggregate metrics across the hierarchy
- Formula fields can reference the parent: Account.Parent.AnnualRevenue

Field name: The standard self-lookup is ParentId (API name), displayed as "Parent Account" on the page. It's just a Lookup field where the referenced object is Account itself.

Important: This is a Lookup, not a Master-Detail. Deleting the parent Account does NOT delete the child Account — the child just loses its parent reference. This is intentional since subsidiaries often outlive their parent company.

---

26. Special Lookups — Significance and Examples

What are special lookups? Certain lookup relationships in Salesforce have special built-in behaviours beyond a standard lookup. They are called "special" because they unlock specific platform features — like Activities, following, and structured hierarchies.

1. What (WhatId) — polymorphic lookup: On Task and Event, the WhatId field is a polymorphic lookup — it can point to any "related-to" object (Account, Opportunity, Case, custom objects). It doesn't point to just one type. The field intelligently understands which object it's pointing to. This is why Tasks can be "related to" an Opportunity, a Case, or a Campaign without needing separate fields.

2. Who (WhoId) — polymorphic lookup: Also on Task and Event. Points to either a Lead or a Contact — whichever person the activity is about. Again, one field that accepts two possible object types.

3. Hierarchical lookup on User: The Manager field on User is a special hierarchical lookup — it's a lookup to User itself but with hierarchy-specific behaviour. It drives the Role Hierarchy reporting chain, $User.Manager formula variables, and approval process "manager" approver type.

4. Account.ParentId: The Parent Account field — a self-lookup (covered above). Enables the Account hierarchy view and cross-hierarchy reporting.

5. Lookup to User: Fields like OwnerId, CreatedById, LastModifiedById are lookups to the User object but are system-managed — read-only except OwnerId (transferable). They power audit trails and record ownership.

Significance: Special lookups enable platform features that regular lookups can't. You cannot replicate WhatId/WhoId behaviour with a custom lookup — it's built into the Activity object architecture.

---

27. Lookup Filters

Definition: A Lookup Filter restricts which records can be selected in a lookup field when a user is populating it. Unlike a Validation Rule (which blocks on save), a Lookup Filter prevents the user from even choosing an invalid related record during entry.

Two types:

- **Required filter** — if the selected record doesn't meet the filter criteria, the save is blocked with an error
- **Optional filter** — applies a search restriction but allows the user to override and select outside the filter with a warning

How to configure: Set up in Object Manager → the child object → Fields → the lookup field → Lookup Filter section. You define criteria using fields on the child record, the parent object's fields, or global variables.

Example configuration:

On Case.AccountId:

Filter: Account.Industry = 'Healthcare'

Type: Required

Error msg: "Please select a Healthcare account for this case."

Result: When the user clicks the lookup icon on AccountId,

only Healthcare accounts appear in search results.

Typing a non-Healthcare account and trying to save is blocked.

Where I would apply it in the NeelamCare project:

On the Program_Patient__c object's Patient__c lookup field (which points to Contact), I would apply a Lookup Filter:

Filter: Contact.RecordType.DeveloperName = 'Patient_RT'

Type: Required

Message: "Only Contacts with the Patient record type can be enrolled in programs."

Why: A Health Worker creating a Program Patient record could accidentally select a Doctor Contact (who also uses the Contact object, just with a different record type). Without this filter, a doctor would be silently enrolled as a patient. The Lookup Filter restricts the search results to only Patient-record-type Contacts, preventing this mistake at the point of entry — before it even reaches a validation rule or trigger.

This is better than a validation rule alone because the filter restricts the SEARCH RESULTS the user sees, guiding them to the right records immediately rather than letting them select the wrong one and then showing an error on save.

Here are in-depth answers to every developer question.

---

1. Data Types Available in Apex

Apex has three categories of data types:

Primitive types — the building blocks:

Type	Size	Example
Integer	32-bit whole number	Integer n = 1000;
Long	64-bit whole number	Long l = 9000000000L;
Double	64-bit float	Double d = 3.14159;
Decimal	Arbitrary precision (default for currency)	Decimal sal = 120000.50;
String	Text	String s = 'Siddhesh';
Boolean	true / false / null	Boolean active = true;
Date	Calendar date	Date today = Date.today();

Type	Size	Example
DateTime	Date + time (stored in UTC)	DateTime now = DateTime.now();
Time	Time of day	Time t = Time.newInstance(10,30,0,0);
Id	Salesforce 15/18-char record identifier	Id accId = '001gK00...';
Blob	Binary data (files, crypto)	Blob b = Blob.valueOf('data');
Object	Generic base type — cast when needed	Object o = 42;

Collections:

```
List<Account>       // ordered, duplicates allowed, index access
Set<Id>             // unordered, no duplicates
Map<Id, Account>    // key-value pairs, unique keys
```

sObject and custom types:

```
Account a = new Account(Name='Acme');  // standard sObject
Enrollment__c e = new Enrollment__c(); // custom sObject
MyClass obj = new MyClass();           // Apex class instance
```

Enum:

```
public enum Season { SPRING, SUMMER, AUTUMN, WINTER }
Season s = Season.SUMMER;
```

null is a valid value for any non-primitive. Accessing a field on a null reference throws NullPointerException.

---

2. Private, Public, and Global Access Modifiers

Private — the most restrictive. Accessible only within the class where it is declared. This is the DEFAULT if no modifier is specified on a method or variable.

```
private Integer salary;          // only accessible inside this class
private void calculateTax() {}   // same
```

Public — accessible from any class within the same namespace (the same Salesforce org or the same package). Use this for most Apex methods and classes that need to be called from other classes in the org.

```
public class AccountService {
    public static List<Account> getAccounts() { ... }
```

```
}

// Callable from any other Apex class in this org
```

Global — the most permissive. Accessible from ANY namespace — including external packages, integrations, and other orgs referencing your managed package. Required for:

- Web services (webservice keyword)
- Methods called by the platform itself (Database.Batchable, Schedulable interfaces)
- Methods in managed packages exposed to subscribers

```
global class MyBatch implements Database.Batchable<sObject> {
    global Database.QueryLocator start(Database.BatchableContext bc) { ... }
    global void execute(Database.BatchableContext bc, List<sObject> scope) { ... }
    global void finish(Database.BatchableContext bc) { ... }
}
```

Protected — accessible within the class and any subclasses that extend it. Used in inheritance hierarchies.

Key rule: You cannot narrow access in a subclass. If the parent declares public, the override must be at least public (can be global, not private).

---

3. Virtual vs Abstract vs Interface

These are three mechanisms for inheritance and abstraction, each with a distinct purpose.

Virtual class: A regular class that CAN be extended and CAN have its methods overridden. It is a complete, instantiable class — you can create objects from it directly. Subclasses use extends and override to customise behaviour.

```
public virtual class Animal {
    public virtual String sound() { return 'Generic sound'; }
    public String breathe() { return 'Inhale, exhale'; }  // non-virtual — can't override
}
public class Dog extends Animal {
    public override String sound() { return 'Woof'; }
}
Animal a = new Animal();  // can instantiate directly
Dog d = new Dog();
```

Abstract class: A class that CANNOT be instantiated directly. It exists only to be subclassed. It can have both abstract methods (no body — subclass MUST implement) and concrete methods (have a body — subclass inherits or overrides). Use it when you want to enforce a contract across subclasses while sharing some common implementation.

```
public abstract class Shape {
    public abstract Double area();          // MUST be overridden
    public String describe() { return 'I am a shape with area ' + area(); }  // shared
}
public class Circle extends Shape {
    private Double radius;
    public Circle(Double r) { this.radius = r; }
    public override Double area() { return Math.PI * radius * radius; }
}
// Shape s = new Shape();  // ERROR — cannot instantiate abstract
Shape c = new Circle(5);   // ok — Circle is concrete
```

Interface: A pure contract — only method signatures, no implementation at all. A class that implements an interface MUST implement every method. A class can implement multiple interfaces (solving Apex's single-inheritance limitation). Interfaces define what something does, not how.

```
public interface Payable { Decimal amount(); String currency(); }
public interface Printable { void print(); }

public class Invoice implements Payable, Printable {
    public Decimal amount() { return 1000.00; }
    public String currency() { return 'INR'; }
    public void print() { System.debug('Invoice: ' + amount() + ' ' + currency()); }
}
```

When to use each:

- Interface — when you have completely different classes that need to share a common API (Database.Batchable, Schedulable are platform interfaces)
- Abstract — when you have a family of related classes sharing some code but requiring subclasses to implement specific behaviour

- Virtual — when you want a full, working class that can optionally be extended and overridden

---

4. Pass by Value vs Pass by Reference in Apex

Primitive types (Integer, String, Boolean, Date, etc.) — passed by VALUE. A copy is made. Modifying the copy inside a method does not affect the original.

```
Integer x = 10;

modifyPrimitive(x);

System.debug(x);  // still 10


static void modifyPrimitive(Integer n) {
   n = 999;   // only modifies the local copy
}
```

Objects, sObjects, collections (List, Set, Map) — passed by REFERENCE. The reference (pointer) is copied, but both the caller and the method point to the same underlying object in heap memory. Mutating the object's contents inside the method DOES affect the original.

```
List<Integer> nums = new List<Integer>{1, 2, 3};

modifyList(nums);

System.debug(nums);  // [1, 2, 3, 999] — the original was modified


static void modifyList(List<Integer> lst) {
   lst.add(999);   // modifies the same list object in memory
}
```

However, reassigning the reference doesn't affect the original:

```
static void replaceList(List<Integer> lst) {
   lst = new List<Integer>{99, 88};  // points to a NEW list; original unchanged
}

// After calling: nums is still [1, 2, 3, 999]
```

This distinction matters in triggers — when you modify Trigger.new records in a before-trigger, you're modifying the actual objects being saved (because they're passed by reference), so no extra DML is needed.

---

5. super and this Keywords

this — refers to the current instance:

```
public class Employee {
    public String name;
    private Decimal salary;

    public Employee(String name, Decimal salary) {
        this.name = name;       // 'this.name' = the field; 'name' alone = the parameter
        this.salary = salary;
    }
    public Employee() {
        this('Unknown', 0);    // calls another constructor in the same class
    }
}
```

this disambiguates when a parameter name shadows a field name. this(...) calls another constructor in the same class (constructor chaining).

super — refers to the parent class:

```
public virtual class Animal {
    public virtual String describe() { return 'I am an Animal'; }
    public Animal(String name) { this.name = name; }
}
public class Dog extends Animal {
    public Dog(String name) {
        super(name);  // calls Animal's constructor — MUST be first statement
    }
    public override String describe() {
        return super.describe() + ' — specifically a Dog';  // calls parent method
    }
```

}

super(...) must be the very first statement in a child constructor. super.method() calls the parent's implementation from inside an override.

---

6. SOQL vs SOSL — When to Use Each

SOQL (Salesforce Object Query Language):

- Queries records from ONE primary object (plus related objects via relationship queries)
- Uses exact field filters — you know the object and the conditions
- Returns a List<sObject> or a single sObject
- Limit: 100 queries / 50,000 rows per transaction
- Best for: "Give me all Accounts where Industry = 'Energy'" — you know exactly which object and what conditions

List<Account> accs = [SELECT Id, Name FROM Account WHERE Industry = 'Energy'];

SOSL (Salesforce Object Search Language):

- Searches indexed text across MULTIPLE objects simultaneously
- Full-text search — finds the keyword wherever it appears in indexed fields
- Returns List<List<sObject>> — one inner list per object type
- Limit: 20 searches / 2,000 rows per transaction
- Best for: "Find 'Acme' wherever it appears — Accounts, Contacts, Opportunities"

List<List<sObject>> results = [

FIND 'Acme*' IN ALL FIELDS

RETURNING Account(Id, Name), Contact(Id, FirstName, LastName)

];

Account[] accs = (Account[]) results[0];

Contact[] cons = (Contact[]) results[1];

Decision guide:

Use SOQL when	Use SOSL when
You know exactly which object to query	You're searching across multiple objects

Use SOQL when	Use SOSL when
You need exact field match filters	You need full-text / keyword search
You're doing DML preparation	User typed a search term in a UI
Reporting, data processing	"Search" functionality

---

7. insert vs Database.insert

insert statement: All-or-none by default. If ANY record in the list fails (validation rule, required field, duplicate), the ENTIRE operation is rolled back and a DmlException is thrown.

```
List<Account> accs = new List<Account>{
    new Account(Name='Acme'),
    new Account()   // Name is required — will fail
};
insert accs;   // DmlException thrown, BOTH records rolled back
```

Database.insert(list, allOrNone): Accepts a second boolean parameter. When false, allows partial success — valid records are saved, invalid ones are rejected, and you get a SaveResult[] array to inspect per-record outcomes.

```
Database.SaveResult[] results = Database.insert(accs, false);
for (Database.SaveResult sr : results) {
    if (sr.isSuccess()) {
        System.debug('Saved: ' + sr.getId());
    } else {
        for (Database.Error e : sr.getErrors()) {
            System.debug('Error: ' + e.getMessage() + ' on fields: ' + e.getFields());
        }
    }
}
```

When to use which:

- insert (all-or-none): when every record must succeed together — transactional integrity is required (e.g., creating a parent + child)

- Database.insert(..., false): when processing a large batch where some failures are acceptable and you want to save as many as possible (e.g., migrating external data where some records may be invalid)

All six DML operations have Database counterparts: Database.update, Database.delete, Database.upsert, Database.undelete, Database.merge.

---

8. Querying Parent and Child in a Single SOQL

Child → Parent (dot notation — going UP):

```
// From Contact, get fields from its parent Account
List<Contact> cons = [
    SELECT FirstName, LastName,
        Account.Name,          // parent field (one dot = one level up)
        Account.Owner.Name     // grandparent field (two dots = two levels up)
    FROM Contact
    WHERE Account.Industry = 'Energy'
];
// Access in code:
for (Contact c : cons) {
    String accName = c.Account.Name;
}
```

Maximum 5 levels up. Custom relationships use __r: Assigned_Doctor__r.Name.

Parent → Child (subquery — going DOWN):

```
// From Account, get all its related Contacts
List<Account> accs = [
    SELECT Id, Name,
        (SELECT Id, FirstName, LastName, Email FROM Contacts),  // standard
        (SELECT Id, Status__c FROM Cases__r)                    // custom: __r
    FROM Account
    WHERE Name = 'Acme'
];
```

```
// Access in code:
for (Account a : accs) {
    List<Contact> childCons = a.Contacts;
    for (Contact c : childCons) {
        System.debug(c.FirstName);
    }
}
```

Child relationship names are plural and use __r suffix for custom objects.

The direction rule (most common exam trap):

- Going DOWN from parent to children = **subquery** with child relationship name
- Going UP from child to parent = **dot notation**

---

9. Governor Limits — Already covered in Admin section

See Admin Question 2 above for the full table. Developer-specific additions:

Why critical in multi-tenant: The same Salesforce infrastructure serves thousands of orgs simultaneously. Without limits, one tenant running a runaway loop could consume all database connections, CPU, and memory, degrading every other tenant's experience. Governor limits are the technical enforcement of fair resource sharing.

Checking limits programmatically:

```
System.debug('SOQL used: ' + Limits.getQueries() + ' of ' + Limits.getLimitQueries());
System.debug('DML used: ' + Limits.getDmlStatements() + ' of ' + Limits.getLimitDmlStatements());
System.debug('CPU used: ' + Limits.getCpuTime() + ' ms of ' + Limits.getLimitCpuTime());
```

---

10. Relationship Queries Limit

When you use a subquery (parent-to-child), Salesforce counts it separately:

- You can have up to **1 subquery per main query** (actually up to 20 subqueries per query in recent API versions, but each subquery counts toward the SOQL query limit)
- Each relationship query counts as a separate SOQL query against the 100/transaction limit
- The records returned by a subquery count toward the 50,000 row limit

```
// This counts as 1 SOQL query but returns parent + children
List<Account> accs = [
    SELECT Id, Name,
        (SELECT Id, LastName FROM Contacts),     // counts as part of this query
        (SELECT Id, Subject FROM Cases)          // also part
    FROM Account
    WHERE Id IN :accIds   // use bind variables to avoid injection
];
```

Practical implication: In a bulkified trigger where you need parent + child data, you run ONE query with a subquery rather than two separate queries — saving your query budget.

---

11. Trigger Context Variables — Complete List

Variable	Available in	Type	Description
Trigger.new	insert (B/A), update (B/A)	List<sObject>	New record values. **Editable in before-context.**
Trigger.old	update (B/A), delete (B)	List<sObject>	Old record values. **Read-only always.**
Trigger.newMap	after insert, update (B/A)	Map<Id,sObject>	Id → new record. Efficient lookup.
Trigger.oldMap	update (B/A), delete (B)	Map<Id,sObject>	Id → old record.
Trigger.isBefore	all	Boolean	True if running in before context.
Trigger.isAfter	all	Boolean	True if running in after context.
Trigger.isInsert	all	Boolean	True on insert.
Trigger.isUpdate	all	Boolean	True on update.
Trigger.isDelete	all	Boolean	True on delete.
Trigger.isUndelete	all	Boolean	True on undelete.

Variable	Available in	Type	Description
Trigger.operationType	all	TriggerOperation enum	e.g. BEFORE_INSERT, AFTER_UPDATE
Trigger.size	all	Integer	Count of records in this batch (max 200).

Trigger.new vs Trigger.old:

- Trigger.new — the records as they WILL BE after the save. On before-insert, these are brand new records with no Id yet. On before-update, these are the modified versions. You CAN modify these in before-context.
- Trigger.old — the records as they WERE before the save. Only available on update and delete. Always read-only. Used to detect field changes: if (a.Status != Trigger.oldMap.get(a.Id).Status).

Not available in certain contexts:

- Trigger.old/Trigger.oldMap — NOT available on insert (there are no old values)
- Trigger.new/Trigger.newMap — NOT available on delete (no new version of a deleted record)
- Trigger.newMap — NOT available on before-insert (Ids don't exist yet)

---

12. Why One Trigger Per Object

The problem with multiple triggers: Salesforce does NOT guarantee execution order among multiple triggers on the same object. If you have Trigger1 and Trigger2 both on Account, Salesforce might run them in any order — and that order can change with each deployment, metadata refresh, or even between transactions.

// Suppose both exist:

Trigger1 on Account → sets Description = 'A'

Trigger2 on Account → sets Description = 'B'

// Which wins? Unpredictable.

What one trigger per object gives you:

1. **Predictable execution order** — you control the sequence via explicit method calls in the handler
2. **Centralized logic** — one place to look when debugging
3. **Bypassing capability** — easy to add a flag to skip the trigger without touching multiple files

4. **Testability** — one test class covers all cases
5. **Scalability** — adding new behaviour = adding a method, not creating a new trigger file

```
// ONE trigger, routes everything
trigger AccountTrigger on Account (before insert, after insert, before update, after update) {
    AccountTriggerHandler h = new AccountTriggerHandler();
    switch on Trigger.operationType {
        when BEFORE_INSERT { h.onBeforeInsert(Trigger.new); }
        when AFTER_INSERT  { h.onAfterInsert(Trigger.new); }
        when BEFORE_UPDATE { h.onBeforeUpdate(Trigger.new, Trigger.oldMap); }
        when AFTER_UPDATE  { h.onAfterUpdate(Trigger.new, Trigger.oldMap); }
    }
}
```

---

13. Trigger Framework

A Trigger Framework is a design pattern separating the trigger entry point from the business logic, making the code testable, maintainable, scalable, and controllable.

Core components:

```
// 1. TRIGGER — thin router (no business logic)
trigger AccountTrigger on Account (before insert, after insert) {
    new AccountTriggerHandler().run();
}

// 2. HANDLER class — all logic, instantiated per invocation
public class AccountTriggerHandler extends TriggerHandler {
    public override void beforeInsert() {
        AccountService.setDefaultDescription(Trigger.new);
    }
    public override void afterInsert() {
        AccountService.createPrimaryContacts(Trigger.new);
```

```
    }
}

// 3. BASE TRIGGER HANDLER — routing + bypass logic
public virtual class TriggerHandler {
    private static Set<String> bypassedHandlers = new Set<String>();
    public static void bypass(String handlerName) { bypassedHandlers.add(handlerName); }
    public static void clearBypass(String n) { bypassedHandlers.remove(n); }

    public void run() {
        if (bypassedHandlers.contains(getHandlerName())) return;
        if (Trigger.isBefore && Trigger.isInsert) this.beforeInsert();
        if (Trigger.isAfter && Trigger.isInsert) this.afterInsert();
        // ... other operations
    }
    protected virtual void beforeInsert() {}
    protected virtual void afterInsert() {}
    private String getHandlerName() { return String.valueOf(this).split(':')[0]; }
}

// 4. SERVICE class — reusable business logic methods
public class AccountService {
    public static void setDefaultDescription(List<Account> accounts) {
        for (Account a : accounts) {
            if (String.isBlank(a.Description)) a.Description = 'Auto-created';
        }
    }
}
```

Why it matters for scalability:

- Bypass without code change: TriggerHandler.bypass('AccountTriggerHandler') — useful in data migration
- Each handler is independently unit-testable
- Adding a new operation = add a method, nothing else changes
- The base class handles routing, freeing handlers to focus on logic

---

14. Recursive Trigger and Prevention

What is a recursive trigger: A trigger that, as part of its execution, performs a DML operation that fires the SAME trigger again — creating an infinite loop until Salesforce kills the transaction.

```
// Scenario: AccountTrigger on after-update → updates Account → fires AccountTrigger again → loop
trigger AccountTrigger on Account (after update) {
    List<Account> toUpdate = new List<Account>();
    for (Account a : Trigger.new) {
        toUpdate.add(new Account(Id=a.Id, Description='Updated'));
    }
    update toUpdate;  // This fires AccountTrigger AGAIN → infinite loop
}
```

Prevention using a static Boolean flag: Static variables live for the ENTIRE transaction (not just one trigger invocation), making them perfect for tracking "has this already run?"

```
// Helper class — static variable persists across trigger invocations in same transaction
public class TriggerHelper {
    public static Boolean accountTriggerHasRun = false;
}


// In the trigger handler:
trigger AccountTrigger on Account (after update) {
    if (TriggerHelper.accountTriggerHasRun) {
        return;  // Already ran — skip this re-entry
    }
```

```
    TriggerHelper.accountTriggerHasRun = true;

    List<Account> toUpdate = new List<Account>();
    for (Account a : Trigger.new) {
        toUpdate.add(new Account(Id=a.Id, Description='Updated'));
    }
    update toUpdate;  // Fires trigger again, but flag is true so it exits immediately
}
```

Why this works: Static variables in Apex are transaction-scoped. Once accountTriggerHasRun is true, it stays true for every subsequent trigger invocation within the same transaction, preventing the loop.

More sophisticated approach — counter-based:

```
public class TriggerHelper {
    public static Integer accountRunCount = 0;
    public static final Integer MAX_RUNS = 1;
}
// In trigger: if (TriggerHelper.accountRunCount >= TriggerHelper.MAX_RUNS) return;
//             TriggerHelper.accountRunCount++;
```

---

15. Salesforce Order of Execution — Complete Detail

When a user clicks Save (or DML is executed), Salesforce follows this exact sequence:

1. Load the original record from the database (on update)
   OR initialise the new record (on insert)

2. System validation:
   - Required field check
   - Data type and format check
   - Maximum field length check
   - Foreign key validation (lookup/master-detail exists)

3. BEFORE TRIGGERS fire (before insert / before update / before delete)
   - You can modify field values on Trigger.new here with no DML
   - Records do NOT have Ids on before-insert yet

4. System validation runs AGAIN + Custom Validation Rules evaluate
   - If any validation rule returns TRUE → error, transaction aborted

5. Duplicate Rules evaluate

6. Record SAVED TO DATABASE (not committed yet)
   → RECORD ID IS GENERATED HERE (insert) or loaded (update)

7. AFTER TRIGGERS fire (after insert / after update / after delete / after undelete)
   - Records have Ids now — you can create related records

8. Assignment Rules (Lead and Case auto-assignment rules)
   → THIS IS WHY a trigger assigning Assigned_Doctor__c in after-insert
    can be overridden — Assignment Rules run AFTER after-triggers

9. Auto-Response Rules (Lead/Case auto-reply emails)

10. Workflow Rules evaluate and fire
   → If workflow contains a field update:
     the record is re-saved, and steps 2-9 re-run ONCE more
     (preventing another infinite loop with the 'once more' cap)

11. Processes (Process Builder) and record-triggered Flows (after-save)
   → Before-save flows actually run at step 3.5 conceptually

12. Escalation Rules

13. Entitlement Rules

14. Roll-Up Summary fields recalculate on the PARENT record

→ This may fire the parent object's triggers and validation rules

15. Cross-Object Formula fields recalculate

16. Sharing Rules evaluated and applied

17. COMMIT TO DATABASE (the transaction becomes permanent)

18. POST-COMMIT LOGIC (runs asynchronously AFTER the commit):

→ Email sends

→ @future methods enqueued

→ Queueable jobs enqueued

→ Platform Events published

Interview-critical insights:

- Before-insert (step 3) runs BEFORE the Id is generated (step 6) — that's why you can't access Trigger.new[0].Id in before-insert
- Validation rules run at step 4 — AFTER before-triggers. So a before-trigger can SET a field before validation runs on it
- Assignment Rules (step 8) run AFTER after-triggers (step 7) — this is the exact bug that affected the NeelamCare DoctorWorkloadBalancer trigger
- @future methods don't run until step 18, after the entire transaction commits

---

16. Testing a Trigger Dependent on a Specific Date or Time

You cannot mock Date.today() or DateTime.now() directly in Apex (unlike some other frameworks). Two strategies:

Strategy 1 — Inject the date via a method parameter:

```
// In production code, pass the date in so tests can control it
public class CaseService {
    public static void flagExpiredCases(List<Case> cases, Date asOfDate) {
        for (Case c : cases) {
            if (c.SLA_Due_Date__c < asOfDate) {
                c.Is_SLA_Breached__c = true;
            }
        }
    }
}

// In test:
@isTest
static void testSLABreach() {
    Case c = new Case(Subject='Test', SLA_Due_Date__c = Date.today().addDays(-1));
    insert c;
    CaseService.flagExpiredCases(new List<Case>{c}, Date.today());
    System.assertEquals(true, [SELECT Is_SLA_Breached__c FROM Case WHERE
Id=:c.Id].Is_SLA_Breached__c);
}
```

Strategy 2 — Test.setFixedSearchResults() for SOSL; for dates, create data relative to today:

```
// Create records with dates relative to today so they're always 'current'
@isTest
static void testScheduledReminder() {
    // Set Next_Due_Date 7 days from now — the flow/trigger checks for THIS_WEEK
    Contact c = new Contact(
        LastName='Patient',
        Next_Due_Date__c = Date.today().addDays(7)
```

```
    );
    insert c;
    // Then invoke the scheduled flow via a Test.startTest/stopTest
    Test.startTest();
    // Execute the scheduled job
    Test.stopTest();
    // Assert email was sent
}
```

Strategy 3 — Use a Date provider class:

```
public class DateService {
    @TestVisible private static Date testDate;
    public static Date today() {
        return testDate != null ? testDate : Date.today();
    }
}
// In test: DateService.testDate = Date.newInstance(2024, 1, 1);
// In code: use DateService.today() instead of Date.today()
```

---

17. Test.startTest() and Test.stopTest()

Two distinct purposes:

Purpose 1 — Fresh governor limits: Any code BEFORE Test.startTest() (typically your @testSetup and data creation) consumes governor limit counts. Without startTest(), those consumptions eat into the limits your code under test gets. After calling startTest(), the governor limit counters RESET — so the code between start and stop gets a fresh 100 SOQL, 150 DML, etc., independent of setup work.

```
@isTest
static void myTest() {
    // This DML and SOQL doesn't count against the code under test
    List<Account> accs = new List<Account>();
    for (Integer k=0; k<50; k++) accs.add(new Account(Name='A'+k));
```

```
    insert accs;  // uses some DML count

    Test.startTest();
    // Fresh limits here — the 150 DML is reset
    MyBatch batch = new MyBatch();
    Database.executeBatch(batch);
    Test.stopTest();  // forces batch to complete

    // Assert results
}
```

Purpose 2 — Force asynchronous code to complete synchronously: Any @future method, System.enqueueJob(), Database.executeBatch(), or scheduled job ENQUEUED inside the startTest/stopTest block is FORCED to complete before stopTest() returns. Without this, async work wouldn't run during the test at all, and your assertions would check data before the job finished.

```
Test.startTest();
System.enqueueJob(new MyQueueable(ids));
// At this point the job is QUEUED but not yet run
Test.stopTest();
// HERE the queueable has been forced to complete synchronously
List<Account> updated = [SELECT Status__c FROM Account WHERE Id IN :ids];
System.assertEquals('Processed', updated[0].Status__c);  // can now assert
```

---

18. 100% Code Coverage for an Exception Block

Exception blocks are not covered if you simply run the happy path — you need to force the exception to be thrown in your test.

Method 1 — Pass intentionally invalid data:

```
// Production code
public static void insertAccount(Account a) {
    try {
        insert a;
```

```
    } catch (DmlException e) {
        System.debug('Failed: ' + e.getMessage());
        // Maybe re-throw or log
    }
}

// Test — pass Account without required Name field
@isTest
static void testInsertAccountException() {
    Account a = new Account();  // Name is required — will throw DmlException
    MyClass.insertAccount(a);   // catch block is now executed and covered
}
```

Method 2 — Force a QueryException:

```
// Production code
try {
    Account a = [SELECT Id FROM Account WHERE Id = :badId LIMIT 1];
} catch (QueryException e) {
    System.debug('Query error: ' + e.getMessage());
}

// Test — pass an Id that doesn't match any record
@isTest
static void testQueryException() {
    MyClass.queryAccount('001000000000000AAA');  // non-existent Id
}
```

Method 3 — Custom exception, throw directly:

```
public class MyException extends Exception {}
public static void validate(Integer n) {
    try {
```

```
        if (n < 0) throw new MyException('Must be positive');
    } catch (MyException e) {
        System.debug(e.getMessage());
    }
}

// Test:
@isTest
static void testNegativeThrowsException() {
    MyClass.validate(-1);  // triggers the catch block
}
```

Key point: Coverage measures which LINES of code execute. The catch block lines only execute when an exception is thrown. You must deliberately cause the exception in your test.

---

19. Passing Parameters from Visualforce to Apex Controller

Method 1 — Controller properties (two-way binding):

```
<!-- Visualforce page -->
<apex:page controller="MyController">
    <apex:form>
        <apex:inputText value="{!searchTerm}" label="Search"/>
        <apex:commandButton value="Search" action="{!doSearch}"/>
        <apex:outputText value="{!result}"/>
    </apex:form>
</apex:page>
public class MyController {
    public String searchTerm { get; set; }  // bound to inputText
    public String result { get; set; }

    public PageReference doSearch() {
```

```
        result = 'Searching for: ' + searchTerm;
        return null;  // stay on page
    }
}
```

Method 2 — URL parameters via ApexPages.currentPage().getParameters():

```
// URL: /apex/MyPage?recordId=001abc&mode=edit
public class MyController {
    public Id recordId { get; private set; }
    public String mode { get; private set; }

    public MyController() {
        recordId = ApexPages.currentPage().getParameters().get('recordId');
        mode = ApexPages.currentPage().getParameters().get('mode');
    }
}
```

Method 3 — Standard Controller extension (for standard objects):

```
public class MyExtension {
    private ApexPages.StandardController sc;

    public MyExtension(ApexPages.StandardController controller) {
        this.sc = controller;
    }
    public void doAction() {
        Account a = (Account) sc.getRecord();  // get the record from the page
        System.debug(a.Name);
    }
}
```

Method 4 — apex:param in action functions:

```
<apex:actionFunction name="processRecord" action="{!process}" rerender="output">
```

```
    <apex:param name="recId" assignTo="{!targetId}" value=""/>
</apex:actionFunction>
<script>processRecord('{!record.Id}');</script>
```

---

20. @future vs Batchable vs Schedulable vs Queueable

Aspect	@future	Batch	Queueable	Schedulable
Use case	Quick async or callout from trigger	Millions of records in chunks	Async with sObjects or job chaining	Run at a specific time
Param types	Primitive / collections of primitive ONLY	None (uses QueryLocator)	Any type including sObjects	None
Monitoring	No	Yes (AsyncApexJob)	Yes (job Id)	Yes
Chaining	No	Via Schedulable	One child job	Via batch enqueue
sObject params	No	N/A	Yes	N/A
Max concurrent	50 queued/trans	5 concurrently	50 queued/trans	100 total
SOQL limit	200	200/execute	200	200

When to choose each:

- **@future** — you're in a trigger context and need to make an HTTP callout. You don't need the result. Simple, one-off async work.
- **Batch** — you must process 10,000+ records. You need start/execute/finish architecture. Memory and CPU pressure from volume requires chunking.
- **Queueable** — you need an async job that accepts sObject parameters, can chain a next step, and can be monitored. The upgrade from @future when you need more capability.
- **Schedulable** — you need something to run at 2am every night. Almost always implemented as a shell that calls Database.executeBatch().

In practice for large-scale work: Schedulable → calls → Batch → optionally chains → another Batch or Queueable.

---

21. @future Limitations

1. **Primitive parameters only** — method must accept primitive types (Integer, String, Boolean, Id) or collections of primitives. You CANNOT pass sObjects.

```
@future
public static void process(Set<Id> contactIds) { ... }  // OK
// process(new List<Contact>{...})  // ERROR — sObjects not allowed
```

2. **Static and void** — must be declared static and return void.
3. **No chaining** — you CANNOT call one @future method from another @future method. Doing so throws a CalloutException ("Future method cannot be called from a future or batch method").
4. **Not monitorable** — once queued, you have no job Id to track it.
5. **Max 50 per transaction** — you can only enqueue 50 @future calls in one transaction.
6. **Can't call from batch execute()** — @future cannot be called from within a Database.Batchable.execute() method.
7. **No guaranteed execution order** — multiple @future calls may execute in any order.

If you need to chain: Use **Queueable** instead, which can enqueue one child job from its execute() method.

---

22. Three Methods of Database.Batchable

```
public class MyBatch implements Database.Batchable<sObject> {

    // 1. START — called ONCE to define the scope of records to process
    public Database.QueryLocator start(Database.BatchableContext bc) {
        // Return a QueryLocator — Salesforce handles pagination automatically
        // Can process up to 50 million records
        return Database.getQueryLocator(
            'SELECT Id, Status FROM Lead WHERE IsConverted = false'
        );
        // OR return an Iterable<sObject> for dynamic/filtered scopes:
```

```
        // return new MyCustomIterable();
    }

    // 2. EXECUTE — called ONCE PER CHUNK (default 200 records, max 2000)
    public void execute(Database.BatchableContext bc, List<Lead> scope) {
        // 'scope' is the current batch of records (up to 200)
        // Full governor limits apply PER invocation of execute()
        for (Lead l : scope) {
            l.Status = 'Processed';
        }
        update scope;
    }

    // 3. FINISH — called ONCE after ALL chunks are complete
    public void finish(Database.BatchableContext bc) {
        // Send summary email, kick off another batch, log completion
        AsyncApexJob job = [SELECT TotalJobItems, NumberOfErrors
                            FROM AsyncApexJob WHERE Id = :bc.getJobId()];
        System.debug('Completed. Chunks: ' + job.TotalJobItems +
                     ', Errors: ' + job.NumberOfErrors);
    }
}
// Invoke: Id jobId = Database.executeBatch(new MyBatch(), 200);
```

BatchableContext bc gives you bc.getJobId() to query the AsyncApexJob record for monitoring.

---

23. Database.Stateful Interface

By default, each call to execute() gets a FRESH instance of the batch class. Any instance variables reset between chunks. Database.Stateful preserves instance variables across all execute() invocations.

```
public class StatefulBatch
    implements Database.Batchable<sObject>, Database.Stateful {

    // This persists across ALL execute() calls
    public Integer totalProcessed = 0;
    public Integer totalErrors = 0;
    public List<String> errorMessages = new List<String>();

    public Database.QueryLocator start(Database.BatchableContext bc) {
        return Database.getQueryLocator('SELECT Id FROM Account');
    }

    public void execute(Database.BatchableContext bc, List<Account> scope) {
        Database.SaveResult[] results = Database.update(scope, false);
        for (Database.SaveResult sr : results) {
            if (sr.isSuccess()) {
                totalProcessed++;
            } else {
                totalErrors++;
                errorMessages.add(sr.getErrors()[0].getMessage());
            }
        }
        // totalProcessed persists to the NEXT execute() call
    }

    public void finish(Database.BatchableContext bc) {
        // All totals are accurate across all chunks
        System.debug('Processed: ' + totalProcessed + ', Errors: ' + totalErrors);
        // Send summary email with errorMessages list
```

```
    }
}
```

Trade-off: Stateful batches use more memory (Salesforce must keep the object alive between chunks). Keep the preserved data lean — don't store thousands of records in instance variables.

---

24. @InvocableMethod — Apex Communication with Flow

@InvocableMethod exposes an Apex method to be called from Flow Builder, making it available as an "Action" element in any flow. It bridges declarative automation with programmatic logic.

```
public class AccountService {


    @InvocableMethod(
        label='Calculate Account Risk Score'
        description='Calculates risk score based on case history'
    )
    public static List<Result> calculateRisk(List<Request> requests) {
        List<Result> results = new List<Result>();
        Set<Id> accIds = new Set<Id>();
        for (Request r : requests) accIds.add(r.accountId);


        Map<Id, Integer> caseCounts = new Map<Id, Integer>();
        for (AggregateResult ar : [
            SELECT AccountId, COUNT(Id) cnt FROM Case
            WHERE AccountId IN :accIds AND IsClosed = false
            GROUP BY AccountId
        ]) {
            caseCounts.put((Id)ar.get('AccountId'), (Integer)ar.get('cnt'));
        }
```

```
        for (Request r : requests) {
            Result res = new Result();
            Integer openCases = caseCounts.containsKey(r.accountId)
                ? caseCounts.get(r.accountId) : 0;
            res.riskScore = openCases > 5 ? 'High' : openCases > 2 ? 'Medium' : 'Low';
            results.add(res);
        }
        return results;
    }

    // Input from Flow
    public class Request {
        @InvocableVariable(required=true label='Account Id')
        public Id accountId;
    }

    // Output to Flow
    public class Result {
        @InvocableVariable(label='Risk Score')
        public String riskScore;
    }
}
```

In Flow Builder:

1. Add an Action element
2. Select "Apex Action"
3. Search for "Calculate Account Risk Score"
4. Map the Account Id input variable and the Risk Score output variable
5. Use the output downstream in the flow

Why it matters: It lets admins use complex Apex logic (aggregate SOQL, calculations, external calls) inside a flow without writing any Apex themselves.

---

25. var vs let vs const — Scoping

Keyword	Scope	Hoisted?	Reassignable?	Re-declarable?
var	**Function-scoped**	Yes (undefined)	Yes	Yes
let	**Block-scoped**	No (TDZ)	Yes	No
const	**Block-scoped**	No (TDZ)	No	No

```
// var — leaks out of blocks, dangerous
function example() {
    if (true) {
        var x = 10;
    }
    console.log(x);  // 10 — x escaped the if-block!
}

// var hoisting
console.log(y);  // undefined (hoisted, not initialised)
var y = 5;

// let — block-scoped, safe
function safe() {
    if (true) {
        let a = 10;
    }
    console.log(a);  // ReferenceError — a is scoped to the if-block
}

// const — block-scoped, can't reassign the binding
```

```
const PI = 3.14;
PI = 3;  // TypeError — can't reassign

const acc = { Name: 'Acme' };
acc.Name = 'Updated';   // ALLOWED — mutating the object is fine
acc = {};               // TypeError — can't reassign the binding itself

// In LWC — use const for components, let for loop variables
const MAX = 200;
for (let i = 0; i < MAX; i++) { /* i scoped to loop */ }
```

Best practice: Use const by default. Switch to let only when you need to reassign the variable. Never use var.

---

26. == vs === in JavaScript

== — Abstract equality (loose comparison with type coercion): JavaScript converts the values to the same type before comparing. This leads to surprising results.

```
0 == false        // true  (false is coerced to 0)
1 == '1'          // true  (string '1' coerced to number 1)
null == undefined // true  (special rule)
'' == false       // true  (both coerce to 0)
[] == 0           // true  (array coerced through string to number)
```

=== — Strict equality (no type coercion): Values must be the same type AND the same value. No surprises.

```
0 === false       // false (different types: number vs boolean)
1 === '1'         // false (different types: number vs string)
null === undefined // false (different types)
1 === 1           // true
'abc' === 'abc'   // true
```

Always use === in JavaScript (including LWC). == is a source of subtle bugs. The only common exception is null == undefined which some developers use to check for "null or undefined" in one expression.

```
// In LWC
handleClick(event) {
    const id = event.target.dataset.id;
    if (id === this.recordId) {  // use ===, not ==
        this.isSelected = true;
    }
}
```

---

27. Aura vs LWC Architecture

Aura (Lightning Components):

- Salesforce-proprietary framework, completely custom
- Runs a heavy JavaScript framework on top of the browser
- Two layers: the Aura framework + your component code
- Events use a proprietary component/application event model
- More verbose syntax (.cmp, .controller.js, .helper.js, .css — separate files per concern)
- Locker Service for security (somewhat strict but not Shadow DOM)

LWC (Lightning Web Components):

- Built on native web standards: HTML5, ES6+, Web Components (W3C standard), Shadow DOM
- The BROWSER does the work — no proprietary abstraction layer needed
- Fewer files: .html, .js, .css, .js-meta.xml
- Standard DOM CustomEvent for communication
- Shadow DOM for style/DOM encapsulation (more strict)
- The engine is smaller, the component loads faster

Why LWC is more performant:

1. Less JavaScript to download and execute — no massive framework overhead
2. Browser handles rendering natively — no virtual DOM reconciliation like in Aura
3. Shadow DOM encapsulation is enforced by the browser engine at native speed
4. Component code is simpler, smaller, faster to parse

5. Reactive property system is built on JavaScript Proxy — highly efficient

Coexistence: An Aura component CAN contain an LWC component (by wrapping it). An LWC component CANNOT contain an Aura component. For all new development, use LWC.

---

28. @api, @track, @wire — Deep Explanation

@api — PUBLIC: Makes a property or method accessible from outside the component (by a parent or the Lightning App Builder). This is the parent-to-child communication channel. Properties become reactive when set by the parent.

```
@api recordId;          // set by the record page or a parent component
@api label = 'Default'; // with a default value
@api refresh() {        // an @api METHOD — parent can call this
    this.loadData();
}
// Parent HTML: <c-child record-id={accId}></c-child>
// Parent JS:   this.template.querySelector('c-child').refresh();
```

Camelcase properties become kebab-case in HTML: recordId → record-id.

@track — DEEP REACTIVITY: Without @track, reassigning a property triggers re-render. With @track, MUTATING the internals of an object or array also triggers re-render. In modern LWC (API 39+), @track is rarely needed because reassignment handles most cases.

```
// Without @track — reassignment works, mutation doesn't
items = [];
this.items = [...this.items, newItem]; // ✅ triggers re-render (new reference)
this.items.push(newItem);              // ❌ doesn't trigger re-render

// With @track — mutation also triggers re-render
@track filters = { city: 'Chennai', active: true };
this.filters.city = 'Mumbai';  // ✅ triggers re-render because @track watches internals
```

@wire — REACTIVE DATA BINDING: Binds a property or method to a Salesforce data source. Re-invokes automatically when $-prefixed reactive parameters change.

```
import getAccounts from '@salesforce/apex/AccountController.getAccounts';
```

```
import { getRecord } from 'lightning/uiRecordApi';

// Property form — result is {data, error}
@wire(getAccounts, { industry: '$selectedIndustry' })
accounts;

// Function form — process the result in a callback
@wire(getAccounts, { industry: '$selectedIndustry' })
handleAccounts({ data, error }) {
    if (data) this.accounts = data;
    if (error) this.error = error;
}

// The $ prefix makes 'selectedIndustry' reactive
// When this.selectedIndustry changes, the wire re-runs automatically
```

---

29. LWC Lifecycle Hooks

Per official Salesforce documentation: *"Lightning web components have a lifecycle managed by the framework. The framework creates components, inserts them into the DOM, renders them, and removes them from the DOM."*

Hook	When	Parent/Child order	Use
constructor()	Instance created	Parent first	super() first; set initial state; NO DOM access, NO @api values
connectedCallback()	Inserted into DOM	Parent first	Subscribe to LMS, fetch data, read recordId
renderedCallback()	After every render	**Child first**	DOM is ready; integrate 3rd-party libs; use boolean flag for one-time
disconnectedCallback()	Removed from DOM	Child first	Unsubscribe LMS, clear timers

Hook	When	Parent/Child order	Use
errorCallback(error, stack)	Descendant throws	—	Error boundary — show fallback UI

```
export default class Demo extends LightningElement {
    hasRendered = false;

    constructor() {
        super();  // MUST be first statement
        // Don't access this.template or @api properties here
        console.log('1. constructor');
    }

    connectedCallback() {
        console.log('2. connectedCallback');
        // Safe to read this.recordId, subscribe to LMS, start data fetch
        this.loadData();
    }

    renderedCallback() {
        console.log('3. renderedCallback — runs after EVERY render');
        if (this.hasRendered) return;  // Guard for one-time operations
        this.hasRendered = true;
        // DOM is ready — can use this.template.querySelector
        const div = this.template.querySelector('div');
    }

    disconnectedCallback() {
        console.log('4. disconnectedCallback');
```

```
        // Clean up — unsubscribe, clear intervals
    }

    errorCallback(error, stack) {
        console.error('5. errorCallback:', error.message);
        // Show a friendly error UI
        this.errorMessage = error.message;
    }
}
```

Hook ordering for parent-child:

```
Parent constructor
Parent connectedCallback
  Child constructor
  Child connectedCallback
  Child renderedCallback    ← children finish first
Parent renderedCallback     ← parent finishes after all children
```

---

30. LMS — Communication Between Unrelated Components

Lightning Message Service (LMS) uses a publish-subscribe pattern over a Message Channel, allowing any component anywhere on the page — even across Aura, VF, and LWC boundaries — to communicate.

Step 1 — Create the Message Channel:

```
<!-- force-app/main/default/messageChannels/PatientSelected.messageChannel-meta.xml -->
<?xml version="1.0" encoding="UTF-8"?>
<LightningMessageChannel xmlns="http://soap.sforce.com/2006/04/metadata">
    <masterLabel>Patient Selected</masterLabel>
    <isExposed>true</isExposed>
    <lightningMessageFields>
        <fieldName>patientId</fieldName>
        <description>The Salesforce Id of the selected patient</description>
```

```
    </lightningMessageFields>
    <lightningMessageFields>
        <fieldName>patientName</fieldName>
    </lightningMessageFields>
</LightningMessageChannel>
```

Step 2 — Publisher component:

```
import { LightningElement, wire } from 'lwc';
import { publish, MessageContext } from 'lightning/messageService';
import PATIENT_CHANNEL from '@salesforce/messageChannel/PatientSelected__c';

export default class PatientList extends LightningElement {
    @wire(MessageContext) messageContext;

    handleRowClick(event) {
        const patientId = event.currentTarget.dataset.id;
        const patientName = event.currentTarget.dataset.name;

        publish(this.messageContext, PATIENT_CHANNEL, {
            patientId: patientId,
            patientName: patientName
        });
    }
}
```

Step 3 — Subscriber component:

```
import { LightningElement, wire } from 'lwc';
import { subscribe, unsubscribe, MessageContext, APPLICATION_SCOPE }
    from 'lightning/messageService';
import PATIENT_CHANNEL from '@salesforce/messageChannel/PatientSelected__c';
```

```
export default class PatientDetail extends LightningElement {
    @wire(MessageContext) messageContext;
    subscription = null;
    patientId;
    patientName;

    connectedCallback() {
        this.subscription = subscribe(
            this.messageContext,
            PATIENT_CHANNEL,
            (message) => this.handleMessage(message),
            { scope: APPLICATION_SCOPE }  // subscribe across the whole app
        );
    }

    disconnectedCallback() {
        unsubscribe(this.subscription);
        this.subscription = null;
    }

    handleMessage(message) {
        this.patientId = message.patientId;
        this.patientName = message.patientName;
    }
}
```

APPLICATION_SCOPE means the subscription works across the entire app; without it, only components in the same region of the page receive the message.

---

31. Imperative vs @wire Apex Call

Aspect	@wire	Imperative
When it runs	Automatically when component renders; re-runs when $-param changes	Only when YOU call it
Control	Framework controls it	You control it completely
DML	Method MUST be cacheable=true — NO DML	DML allowed
Error handling	{data, error} object	.then()/.catch() or try/catch
Caching	LDS client-side cache	No automatic caching
Use for	Read-only display data that should refresh reactively	User actions, writes, sequential operations
Refresh	Need refreshApex(this.wiredResult)	Just call the method again

```
// @wire — set and forget, re-runs on parameter change
@wire(getContacts, { accId: '$recordId' })
wiredContacts; // {data, error} - framework manages it

// IMPERATIVE — you decide when to run
async loadContacts() {
    try {
        this.contacts = await getContacts({ accId: this.recordId });
    } catch (error) {
        this.error = error.body.message;
    }
}

// Button handler — imperative is correct here
handleSave() {
    saveContact({ contact: this.contact })
        .then(() => {
            this.showToast('success', 'Saved!');
```

```
            return refreshApex(this.wiredContacts); // refresh wire after DML
        })
        .catch(error => this.showToast('error', error.body.message));
}
```

Scenario Questions

32. Update 500,000 Lead records

Design: Batch Apex + Schedulable

```
public class LeadStatusBatch implements Database.Batchable<sObject>, Database.Stateful {
    private String newStatus;
    private Integer successCount = 0;
    private Integer errorCount = 0;

    public LeadStatusBatch(String status) { this.newStatus = status; }

    public Database.QueryLocator start(Database.BatchableContext bc) {
        // QueryLocator handles up to 50M records
        return Database.getQueryLocator(
            'SELECT Id, Status FROM Lead WHERE Status != :newStatus'
        );
    }

    public void execute(Database.BatchableContext bc, List<Lead> scope) {
        for (Lead l : scope) l.Status = newStatus;

        // Use Database.update(false) for partial success — don't fail the whole chunk
        Database.SaveResult[] results = Database.update(scope, false);
        for (Database.SaveResult sr : results) {
            if (sr.isSuccess()) successCount++;
```

```
            else errorCount++;
        }
    }

    public void finish(Database.BatchableContext bc) {
        Messaging.SingleEmailMessage mail = new Messaging.SingleEmailMessage();
        mail.setToAddresses(new List<String>{'admin@company.com'});
        mail.setSubject('Lead Update Complete');
        mail.setPlainTextBody('Updated: ' + successCount + ', Errors: ' + errorCount);
        Messaging.sendEmail(new List<Messaging.SingleEmailMessage>{mail});
    }
}
// Run: Database.executeBatch(new LeadStatusBatch('Working'), 2000);
// chunk size 2000 = max for batch (reduces number of chunks = fewer DB transactions)
```

Why this design:

- Batch processes in chunks of up to 2,000, each with its own fresh governor limits
- Database.Stateful tracks totals across all chunks
- Database.update(scope, false) allows partial success — one bad record doesn't kill the chunk
- QueryLocator can handle 50M records vs Iterable (50K)

---

33. Notify Third-Party AFTER Transaction Commits

The key constraint: "after the transaction is committed." This means synchronous Apex or even after-triggers won't work — they run BEFORE commit. The solution is **@future or Platform Events.**

Solution using @future:

```
// In the Trigger — runs during the transaction
trigger OpportunityTrigger on Opportunity (after update) {
    List<Id> closedIds = new List<Id>();
    for (Opportunity o : Trigger.new) {
```

```
        Opportunity old = Trigger.oldMap.get(o.Id);
        if (o.StageName == 'Closed Won' && old.StageName != 'Closed Won') {
            closedIds.add(o.Id);
        }
    }
    if (!closedIds.isEmpty()) {
        ThirdPartyNotifier.notifyAsync(closedIds);
        // @future is queued but NOT executed yet
    }
}

public class ThirdPartyNotifier {
    @future(callout=true)
    public static void notifyAsync(List<Id> oppIds) {
        // This runs AFTER the transaction commits (Step 18 of OoE)
        for (Id oppId : oppIds) {
            HttpRequest req = new HttpRequest();
            req.setEndpoint('callout:ThirdParty_Named_Credential/notify');
            req.setMethod('POST');
            req.setBody('{"opportunityId":"' + oppId + '"}');
            HttpResponse res = new Http().send(req);
            // handle response
        }
    }
}
```

Why @future is correct here: @future methods enqueue at Step 17 and execute at Step 18 — AFTER the transaction commits (Step 17). So the third-party system is only called after the Opportunity data is permanently in the database.

Alternative — Platform Events: Publish a platform event in the trigger → subscribe externally via webhook/CometD. Even more decoupled and more reliable for guaranteed delivery.

---

34. Account Address Update Cascades to Contacts with Full Rollback

Requirement: Account updates → all related Contacts update. If any ONE Contact fails → roll back EVERYTHING including the Account.

```
trigger AccountTrigger on Account (after update) {
   AccountTriggerHandler.handleAfterUpdate(Trigger.new, Trigger.oldMap);
}


public class AccountTriggerHandler {
   public static void handleAfterUpdate(List<Account> newList, Map<Id,Account> oldMap)
{
      List<Id> changedIds = new List<Id>();
      for (Account a : newList) {
         if (a.BillingStreet != oldMap.get(a.Id).BillingStreet ||
            a.BillingCity   != oldMap.get(a.Id).BillingCity   ||
            a.BillingState  != oldMap.get(a.Id).BillingState  ||
            a.BillingCountry!= oldMap.get(a.Id).BillingCountry) {
            changedIds.add(a.Id);
         }
      }
      if (changedIds.isEmpty()) return;

      Map<Id, Account> accMap = new Map<Id, Account>(
         [SELECT Id, BillingStreet, BillingCity, BillingState, BillingCountry
          FROM Account WHERE Id IN :changedIds]
      );
```

```
        List<Contact> toUpdate = [
            SELECT Id, AccountId, MailingStreet FROM Contact
            WHERE AccountId IN :changedIds
        ];

        for (Contact c : toUpdate) {
            Account a = accMap.get(c.AccountId);
            c.MailingStreet  = a.BillingStreet;
            c.MailingCity    = a.BillingCity;
            c.MailingState   = a.BillingState;
            c.MailingCountry = a.BillingCountry;
        }

        // If ANY Contact update fails, the whole transaction (including Account) rolls back
        // Because this DML is in the SAME transaction as the Account update
        update toUpdate;  // all-or-none DML statement — one failure = full rollback
    }
}
```

Why this provides full rollback: The trigger runs in the SAME DATABASE TRANSACTION as the Account update. update toUpdate is all-or-none. If any Contact fails, it throws a DmlException which is uncaught, causing the ENTIRE transaction — including the Account save — to roll back to its pre-save state.

If you want to inspect and handle errors without a full rollback: use Database.update(toUpdate, false), check the results, and manually call addError() on the Account if contacts failed — which then rolls back the Account too.

---

35. Enforce FLS in Apex

By default, Apex runs in system mode and IGNORES Field-Level Security. You must explicitly enforce it.

Method 1 — WITH SECURITY_ENFORCED in SOQL:

```
try {
```

```
    List<Account> accs = [
        SELECT Id, Name, AnnualRevenue, SSN__c   // if user can't see SSN__c, throws
        FROM Account
        WITH SECURITY_ENFORCED
    ];
} catch (System.QueryException e) {
    // User lacks access to one or more fields
    System.debug('FLS violation: ' + e.getMessage());
}
```

Throws System.QueryException if the running user doesn't have read access to any queried field.

Method 2 — Security.stripInaccessible() (more flexible):

```
List<Account> accs = [SELECT Id, Name, AnnualRevenue FROM Account];

// Strip fields the user can't READ before returning to LWC
SObjectAccessDecision readDecision =
    Security.stripInaccessible(AccessType.READABLE, accs);
List<Account> safeAccs = (List<Account>) readDecision.getRecords();

// Strip fields the user can't EDIT before DML
SObjectAccessDecision editDecision =
    Security.stripInaccessible(AccessType.UPDATABLE, accs);
update (List<Account>) editDecision.getRecords();
```

Method 3 — Manual Schema check:

```
public static Boolean canReadField(SObjectType objType, String fieldName) {
    return objType.getDescribe()
                .fields.getMap()
                .get(fieldName)
                .getDescribe()
```

```
            .isAccessible();
}

if (!canReadField(Account.sObjectType, 'AnnualRevenue')) {
   throw new SecurityException('You do not have access to Annual Revenue.');
}
```

For the specific scenario (calculation fails if field is inaccessible):

```
@AuraEnabled
public static Decimal calculateProfit(Id accId) {
   // Check FLS before accessing sensitive field
   if (!Schema.sObjectType.Account.fields.Profit_Margin__c.isAccessible()) {
      throw new AuraHandledException(
         'You do not have permission to view Profit Margin.'
      );
   }
   Account a = [SELECT Id, Revenue__c, Cost__c, Profit_Margin__c
            FROM Account WHERE Id = :accId WITH SECURITY_ENFORCED];
   return a.Revenue__c * a.Profit_Margin__c;
}
```

---

36. Wire Doesn't Refresh After Imperative Call

The problem: @wire caches results on the client. After you do a DML update via an imperative call, the wire still shows the old cached data.

Solution: refreshApex()

```
import { LightningElement, wire } from 'lwc';
import { refreshApex } from '@salesforce/apex';
import getContacts from '@salesforce/apex/ContactController.getContacts';
import saveContact from '@salesforce/apex/ContactController.saveContact';
```

```
export default class ContactManager extends LightningElement {
    _wiredResult;  // capture the wire result for refreshApex

    @wire(getContacts, { accId: '$recordId' })
    handleWire(result) {
        this._wiredResult = result;  // MUST store the full {data, error} object
        if (result.data) this.contacts = result.data;
        if (result.error) this.error = result.error;
    }

    async handleSave() {
        try {
            await saveContact({ contact: this.contactToSave });
            // After the imperative DML succeeds, force the wire to re-fetch
            await refreshApex(this._wiredResult);
            // Now this.contacts shows the updated data
            this.showToast('success', 'Saved');
        } catch (error) {
            this.showToast('error', error.body.message);
        }
    }
}
```

Key points:

- You must store the FULL {data, error} object (not just result.data) for refreshApex to work
- refreshApex bypasses the LDS cache and makes a fresh server request
- For LDS-managed components (lightning-record-form, getRecord), the cache is automatically invalidated on save — no refreshApex needed

---

37. Infinite Loop Between Trigger A and Trigger B

Problem:

Trigger on Object A → updates Object B → fires Trigger on Object B → updates Object A → fires Trigger on Object A → infinite loop

Solution: Cross-object static Boolean guards

```
// Single shared control class
public class TriggerControl {
    public static Boolean aUpdatingB = false;
    public static Boolean bUpdatingA = false;
}

// Trigger on Object A
trigger ObjectATrigger on Object_A__c (after update) {
    if (TriggerControl.bUpdatingA) return;  // skip if B caused this
    TriggerControl.aUpdatingB = true;
    try {
        // ... update Object B records ...
    } finally {
        TriggerControl.aUpdatingB = false;
    }
}

// Trigger on Object B
trigger ObjectBTrigger on Object_B__c (after update) {
    if (TriggerControl.aUpdatingB) return;  // skip if A caused this
    TriggerControl.bUpdatingA = true;
    try {
        // ... update Object A records ...
    } finally {
        TriggerControl.bUpdatingA = false;
```

```
    }
}
```

Why it works:

- When Object A's trigger sets aUpdatingB = true and updates B, Object B's trigger sees the flag is true and exits immediately
- The finally block ensures the flag is reset even if an exception occurs
- Two separate flags let you distinguish which direction the update came from

---

38. Child Tells Parent That an Item Was Clicked (CustomEvent)

```
// CHILD — resultList.js
import { LightningElement, api } from 'lwc';

export default class ResultList extends LightningElement {
    @api results;  // passed down from parent (Search component)

    handleItemClick(event) {
        const itemId = event.currentTarget.dataset.id;
        const itemName = event.currentTarget.dataset.name;

        // Dispatch a custom event UP to the parent
        this.dispatchEvent(new CustomEvent('itemselect', {
            detail: {
                id: itemId,
                name: itemName
            },
            bubbles: false,    // don't propagate up the DOM
            composed: false    // stay within Shadow DOM boundary
        }));
    }
```

```
}
<!-- resultList.html -->
<template>
    <template for:each={results} for:item="r">
        <div key={r.Id}
             data-id={r.Id}
             data-name={r.Name}
             onclick={handleItemClick}
             class="result-item">
            {r.Name}
        </div>
    </template>
</template>
// PARENT — search.js
export default class Search extends LightningElement {
    results = [];
    selectedId;
    selectedName;

    handleSearch(event) {
        // ... perform search, populate this.results
    }

    // Listen for the 'itemselect' event from the child
    handleItemSelect(event) {
        this.selectedId = event.detail.id;
        this.selectedName = event.detail.name;
        console.log('Selected:', this.selectedId, this.selectedName);
    }
```

```
}
<!-- search.html -->
<template>
    <lightning-input label="Search" onchange={handleSearch}></lightning-input>

    <!-- Listen to 'itemselect' by prefixing with 'on' -->
    <c-result-list
        results={results}
        onitemselect={handleItemSelect}>
    </c-result-list>

    <p>Selected: {selectedName}</p>
</template>
```

Rule: Child dispatches new CustomEvent('eventname', { detail: {...} }). Parent listens with on + eventname = onitemselect.

---

39. Change Set Fails Due to Missing Dependencies

Why it happens: A Change Set only includes what you explicitly added. If a Validation Rule references a Custom Field, or a Flow uses an Apex class, those dependencies must ALSO be in the Change Set — Salesforce doesn't auto-include them.

How to identify missing dependencies:

1. Open the Outbound Change Set in the source org
2. Click **View/Add Dependencies** — Salesforce scans your selected components and shows what else must be included
3. Add ALL listed dependencies to the Change Set
4. In the target org after a failed deployment, check the **Deployment Errors** detail — it names exactly which component is missing

Step-by-step fix:

Source Org:

1. Setup → Deploy → Outbound Change Sets → Open your set

2. Click 'View/Add Dependencies'

3. Salesforce lists all unresolved dependencies

4. Select all → Add to Change Set

5. Upload again

Target Org (if it failed):

1. Setup → Deploy → Deployment Status

2. Click on the failed deployment

3. Read the error detail: "Component X requires Component Y which is missing"

4. Go back to source org, add Y to the Change Set

Best practices to prevent this:

- Always click "View/Add Dependencies" BEFORE uploading
- Use sf project deploy validate with the Metadata API to validate before deploying
- Keep a deployment checklist: Custom Object → Fields → Page Layouts → Validation Rules → Flows → Apex Classes → Profiles
- Prefer SFDX source-driven deployment over Change Sets for complex projects — it validates the entire package including dependencies

---

40. Search "Salesforce" Across Accounts, Contacts, and Opportunities

This is exactly what **SOSL** is designed for — searching across multiple objects simultaneously.

```
// SOSL across three objects
List<List<sObject>> results = [
    FIND '"Salesforce"'          // exact phrase match with quotes
    IN ALL FIELDS                // search Name, Description, and all text fields
    RETURNING
        Account(Id, Name, Industry WHERE IsDeleted = false),
        Contact(Id, FirstName, LastName, Email),
        Opportunity(Id, Name, StageName, Amount)
    LIMIT 200                    // max 200 total results
];
```

```
Account[]     accounts     = (Account[])     results[0];
Contact[]     contacts     = (Contact[])     results[1];
Opportunity[] opportunities = (Opportunity[]) results[2];

System.debug('Accounts found: ' + accounts.size());
System.debug('Contacts found: ' + contacts.size());
System.debug('Opportunities found: ' + opportunities.size());
```

LWC imperative version for a search bar:

```
import searchRecords from '@salesforce/apex/SearchController.searchAcrossObjects';

handleSearch(event) {
    const keyword = event.target.value;
    if (keyword.length < 3) return;  // don't search on single characters
    searchRecords({ keyword: keyword })
        .then(result => {
            this.accounts     = result.accounts;
            this.contacts     = result.contacts;
            this.opportunities = result.opportunities;
        })
        .catch(error => this.error = error.body.message);
}
@AuraEnabled(cacheable=true)
public static Map<String, List<sObject>> searchAcrossObjects(String keyword) {
    String query = '*' + String.escapeSingleQuotes(keyword) + '*';
    List<List<sObject>> raw = Search.query(
        'FIND {' + query + '} IN ALL FIELDS RETURNING ' +
        'Account(Id,Name), Contact(Id,FirstName,LastName), Opportunity(Id,Name)'
    );
```

```
    return new Map<String, List<sObject>>{
        'accounts' => raw[0],
        'contacts' => raw[1],
        'opportunities' => raw[2]
    };
}
```

---

41. Batch Job Fails at 5,000th Record — What Happens to First 4,999?

Answer: It depends on whether you used Database.Batchable with partial success or all-or-none DML.

Scenario A — Using DML statement (update scope;) — all-or-none per chunk: Each execute() call processes one chunk (e.g. 200 records). Each chunk is its own transaction. If the 5,000th record is in chunk 25 (records 4801-5000), only THAT CHUNK rolls back. All previous chunks (chunks 1-24, records 1-4800) are already COMMITTED and unaffected.

Chunk 1 (1-200): ✅ Committed

Chunk 2 (201-400): ✅ Committed

...

Chunk 24 (4601-4800): ✅ Committed

Chunk 25 (4801-5000): ❌ LimitException → THIS CHUNK rolls back

Chunk 26+ (5001+): Never executed

Scenario B — Using Database.update(scope, false) (partial success): Even within the failing chunk, the 199 successful records are saved. Only the specific record causing the LimitException fails. The batch continues to the next chunk.

The real gotcha — LimitException in the execute() method: If the LimitException occurs because of hitting CPU/SOQL/DML limits WITHIN execute(), Salesforce marks that chunk as failed in AsyncApexJob.NumberOfErrors, logs the error, and moves to the next chunk. It does NOT stop the entire batch job.

How to handle:

```
public void execute(Database.BatchableContext bc, List<Lead> scope) {
    try {
        Database.SaveResult[] results = Database.update(scope, false);
        // Log per-record errors without failing the chunk
```

```
        for (Database.SaveResult sr : results) {
            if (!sr.isSuccess()) errorCount++;
        }
    } catch (Exception e) {
        // Log error for this chunk, batch continues to next chunk
        System.debug('Chunk failed: ' + e.getMessage());
        errorCount++;
    }
}
```

---

42. list.add() and update Inside a for Loop — Why It Fails for 200

The code with the problem:

```
// BROKEN CODE
for (Account a : Trigger.new) {
    Contact c = [SELECT Id FROM Contact WHERE AccountId = :a.Id LIMIT 1]; // SOQL in loop
    c.Description = 'Updated';
    update c;  // DML in loop
}
```

Why it works for 50 records but fails for 200:

For 50 records:

- SOQL queries used: 50 (under the 100 limit ✅)
- DML statements used: 50 (under the 150 limit ✅)

For 200 records:

- SOQL queries used: 200 → LimitException at record 101 ❌
- Even if SOQL survived, DML would fail at record 151 ❌

The fix — bulkify using collections and single DML:

```
// CORRECT CODE
```

```
// Step 1: Collect all Account Ids first
Set<Id> accountIds = new Set<Id>();
for (Account a : Trigger.new) {
    accountIds.add(a.Id);
}

// Step 2: ONE query to get all needed Contacts
Map<Id, Contact> contactsByAccount = new Map<Id, Contact>();
for (Contact c : [SELECT Id, AccountId FROM Contact WHERE AccountId IN :accountIds])
{
    contactsByAccount.put(c.AccountId, c);
}

// Step 3: Loop in MEMORY — no SOQL or DML inside
List<Contact> toUpdate = new List<Contact>();
for (Account a : Trigger.new) {
    if (contactsByAccount.containsKey(a.Id)) {
        Contact c = contactsByAccount.get(a.Id);
        c.Description = 'Updated for ' + a.Name;
        toUpdate.add(c);
    }
}

// Step 4: ONE DML outside the loop
if (!toUpdate.isEmpty()) update toUpdate;

// For 200 records: 1 SOQL + 1 DML = way within limits ✅
```

The mental model:

- COLLECT in loop (just array operations — free)

- QUERY once outside loop (1 SOQL regardless of batch size)
- BUILD update list in loop (free)
- DML once outside loop (1 DML regardless of batch size)

This pattern scales to any number of records limited only by the 50,000-row SOQL return limit and 10,000-row DML limit — both far beyond the 200-record trigger batch size.